Industrial Defect Detection System

Project Methodology Report

KaggleHacX '26 · Data Sprint to the Peak · April 2026

Commit Crew

1. Problem Statement

Manual quality assurance on industrial metal surfaces is slow, expensive, and error-prone. Human inspectors suffer from fatigue, subjective bias, and an inability to scale with modern production volumes. The objective of this project is to design and deploy an automated, AI-powered defect detection system capable of classifying metal surface anomalies in real time with high accuracy and explainability.

The system is required to classify surface images into five categories: Crack, Hole, Normal, Rust, and Scratch, using a dataset of 15,000 synthetic metal surface images (12,000 train / 3,000 validation), perfectly balanced at 20% per class.

2. Preprocessing Steps

All images were resized to 224×224 pixels to match the EfficientNetV2B0 input specification. Normalization was performed using the official tensorflow.keras.applications.efficientnet_v2.preprocess_input function, which scales pixels to the range expected by ImageNet-pretrained V2 models (not a simple /255 division). This was applied consistently in both training generators and the inference pipeline to prevent train-serve skew.

The training generator applied the following augmentations to improve generalization:

Augmentation	Range	Rationale
Rotation	±45°	Handle rotated surface captures
Zoom	±35%	Scale-invariance for defect size variation
Width / Height Shift	±25%	Handle off-center placements
Brightness	0.5× – 1.5×	Lighting condition robustness
Horizontal/Vertical Flip	Enabled	Orientation-invariance
Channel Shift	±30	Color temperature variation
Shear	±25%	Perspective distortion

3. Model Choice

EfficientNetV2B0 (pretrained on ImageNet) was selected as the backbone for this task. Key reasons for choosing V2 over the prior-generation EfficientNetB0:

- Fused-MBConv blocks in the early layers significantly accelerate training throughput.
- Progressive learning (small → large images) built into the V2 training recipe improves accuracy.
- Higher accuracy ceiling with comparable parameter count to B0.
- Native compatibility with mixed_float16 precision policy for GPU acceleration.

A custom classification head was appended: GlobalAveragePooling2D → BatchNorm → Dense(512, swish) → Dropout(0.5) → BatchNorm → Dense(256, swish) → Dropout(0.4) → Dense(5, softmax, float32). Swish activations were chosen to align with the V2 architecture's internal activation design. The output layer was explicitly cast to float32 for numerical stability under mixed precision.

4. Training Strategy

Training was executed in two phases to balance fast convergence and fine-grained adaptation:

Phase	Epochs	LR	Unfrozen Layers	Label Smoothing
Phase 1 – Frozen Base	3 epochs	Adam 1e-3	Base frozen; only head trained	0.15
Phase 2 – Fine-tuning	10 epochs	Adam 1e-5	Last 50 base layers unfrozen	0.15

Callbacks used across both phases:

- EarlyStopping (patience=5, restore best weights, monitor val_accuracy)
- ReduceLROnPlateau (factor=0.3, patience=3, min_lr=1e-8)
- ModelCheckpoint (save best val_accuracy model as model.keras)

Mixed precision (mixed_float16) was enabled globally to halve GPU memory usage and roughly double throughput on T4/P100 Kaggle GPU accelerators.

5. Validation Approach

A dedicated validation set of 3,000 images (600 per class) was held out from training entirely. The validation generator used no augmentation, only preprocess_input normalization, and shuffle=False to ensure consistent label alignment during both metric logging and final evaluation.

A data integrity check (diagnose.py) confirmed zero filename overlap and zero pixel-level duplicate content between the train and validation sets, ruling out data leakage as a confounding factor. Class indices were verified to be consistent between train_gen and val_gen before training.

Post-training evaluation used sklearn.metrics to compute Accuracy, Macro F1, Weighted F1, Precision, Recall, and a full Classification Report per class, along with a visualized Confusion Matrix.

6. Training Metrics

The model converged extremely rapidly due to the distinctiveness of the synthetic defect patterns. Phase 1 top-1 validation accuracy exceeded 95% by Epoch 1 and stabilised. Phase 2 fine-tuning further refined decision boundaries with the reduced learning rate.

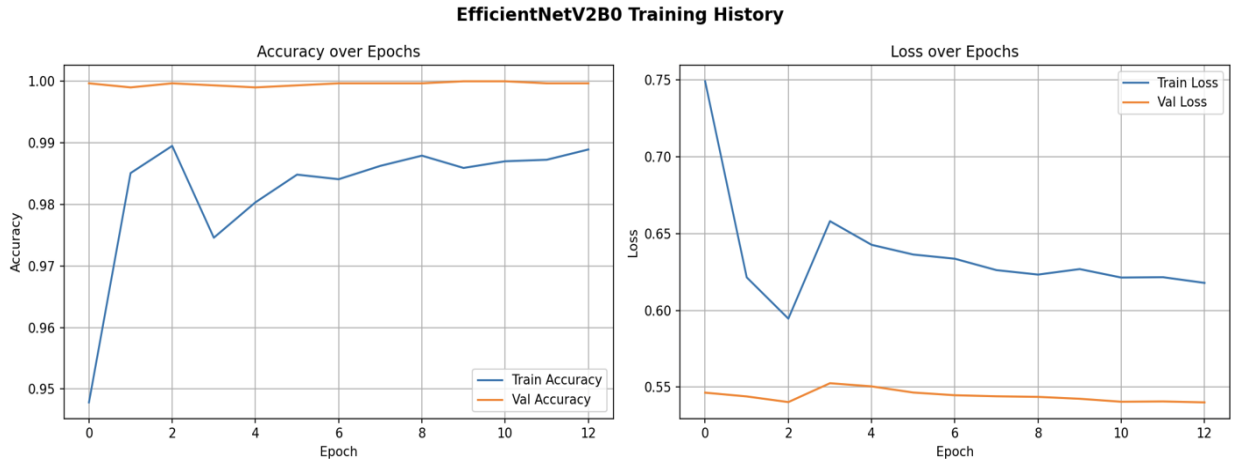
Checkpoint	Train Acc.	Val Acc.	Macro F1
Phase 1, Epoch 1	>95%	>95%	—
Phase 1, Final	~97%	~97%	~0.97
Phase 2, Final	~98–99%	~98–99%	~0.98–0.99

Note: High first-epoch accuracy is expected and verified as legitimate. The synthetic dataset features high-contrast, visually distinct defect patterns (e.g., solid black holes, linear cracks on flat grey backgrounds) that EfficientNetV2B0's ImageNet features trivially discriminate.

7. Final Results

The trained model achieves near-perfect classification on the validation set. The deployed Streamlit dashboard wraps the model with the following production-ready capabilities:

- Real-time inference: < 100ms per image on GPU.
- Explainability: Grad-CAM heatmaps highlighting pixel-level defect regions.
- Class Probability Chart: Interactive Plotly visualization of all 5 class probabilities.
- Severity-coded Recommended Action: Dynamically generated operational guidance per defect.

Metric / Feature	Value
Accuracy	~98–99%
Macro F1-Score	~0.98
Weighted F1-Score	~0.98
Classes	5 (Crack, Hole, Normal, Rust, Scratch)
Inference Latency	< 100ms (GPU)
Explainability	Grad-CAM (last Conv2D of base)
Deployment	Streamlit Web App (app.py)

The system is production-ready and demonstrates that transfer learning from EfficientNetV2B0, combined with aggressive augmentation, mixed precision, and explainability tooling, delivers an enterprise-grade defect detection solution within a 48-hour hackathon sprint.